



**NANYANG
TECHNOLOGICAL
UNIVERSITY**
SINGAPORE

Centre for Professional and
Continuing Education

Introduction to Object-Oriented Programming (OOP)



Object-Oriented Principles Part 1

Lesson Objectives

By the end of this lesson, students will:

- Understand the purpose of classes and objects
- Create and use constructors (parameterized, overloaded, copy)
- Apply encapsulation with private fields, getters, and setters
- Differentiate instance and static members
- Override toString() for better object representation
- Model data using POJOs and records



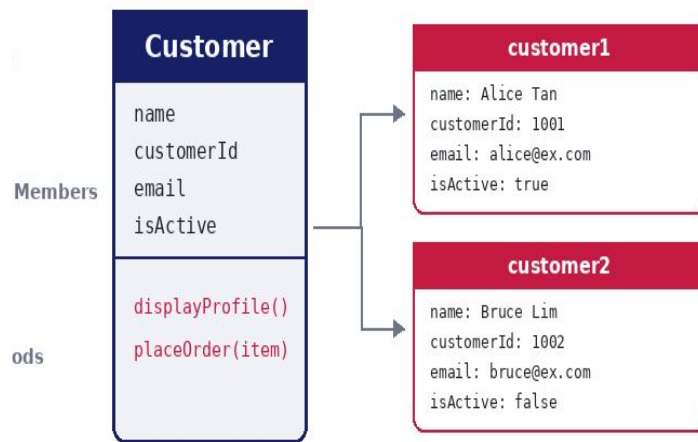
Introduction to Classes

A class is a blueprint for creating objects.

Objects have attributes (data) and behaviors (methods).

Classes help in organizing and structuring code efficiently.

Java Class & Objects



Defining a Class

Class: is a blueprint/template that describes what data (attributes) and behaviors (methods) an object has.

(methods) an object has:

In simpler terms, an object is a specific instance of a class, containing attributes (data) and methods (behaviors) that define its properties and actions. For example, in a class Customer, an object could be customer1 with attributes like name, customerId, and isActive, and methods like displayProfile()



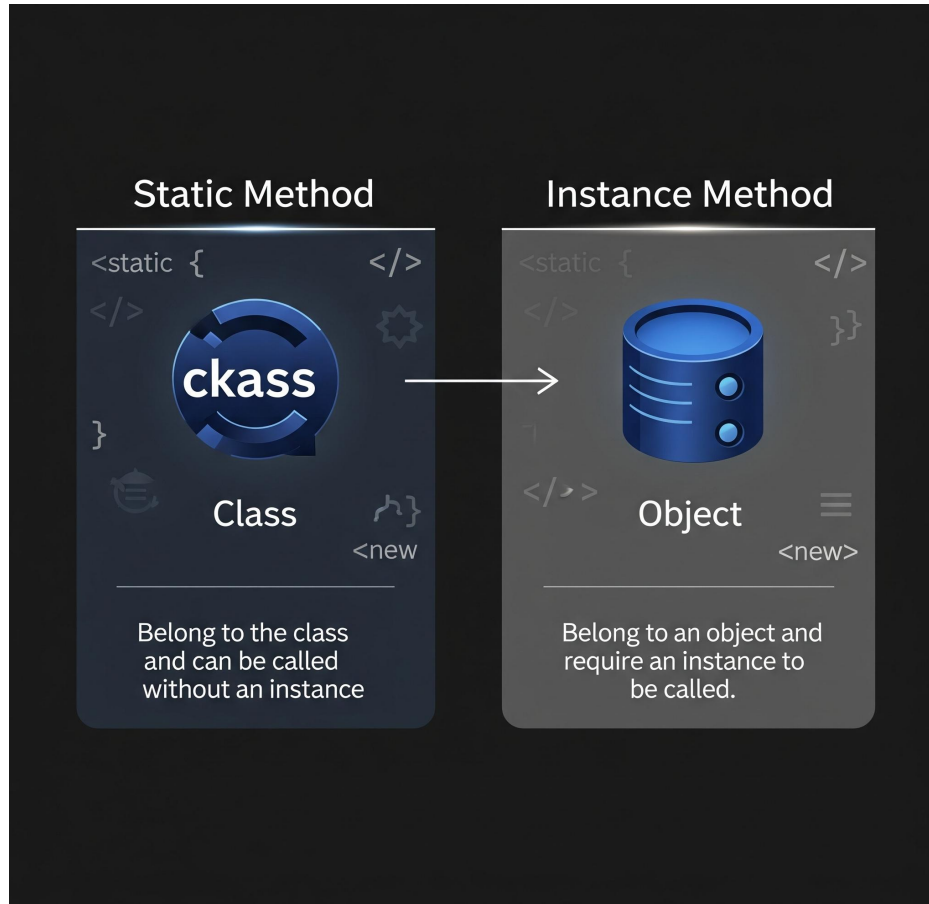
```
public class Customer {  
  
    // Attributes (state)  
    String name;  
    int customerId;  
    String email;  
    boolean isActive;  
  
    // Methods (behaviour)  
    public void displayProfile() {  
        System.out.println("Customer #" + customerId  
            + ": " + name + " (" + email + ")");  
    }  
  
    public void placeOrder(String item) {  
        System.out.println(name + " placed an order for: " + item);  
    }  
}
```

Creating and Using Objects

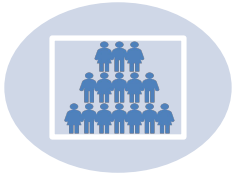


```
public class App {  
    public static void main(String[] args) {  
        // Creating an object of the Customer class  
        Customer c1 = new Customer();  
  
        // Setting attribute values  
        c1.name = "Alice Tan";  
        c1.customerId = 1001;  
        c1.email = "alice@example.com";  
        c1.isActive = true;  
  
        // Calling the method  
        c1.displayProfile();  
        // Output: Customer #1001: Alice Tan (alice@example.com)  
    }  
}
```

Static Vs Instance Method



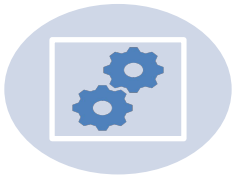
Summary



- Classes help group related attributes and behaviors.



- Objects are instances of a class.



- Methods define behaviors, and attributes store state.



- Static methods belong to the class, instance methods belong to objects.

What is a Constructor?

- A constructor is a special method called automatically
- It executes immediately upon creating a new object
- This method is used primarily to initialize object attributes
- The constructor must have the exact same name as the class

Constructor

Rules

- Same name as the class
- No return type at all
- Runs automatically on new
- Initialises the attributes

Class: Customer

```
Customer(name,  
customerId,  
email, isActive)
```

blueprint

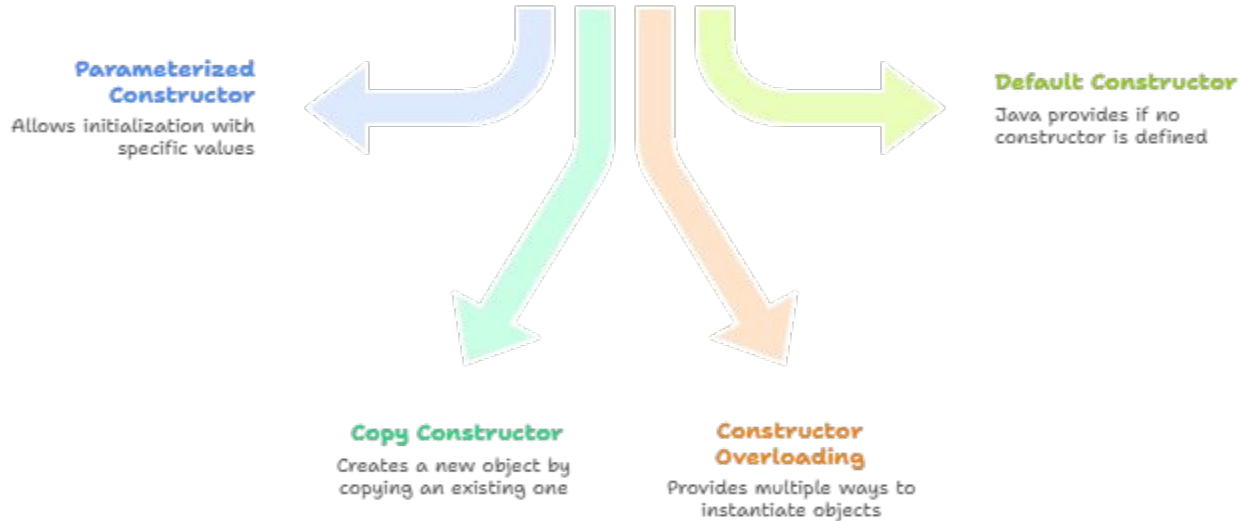


Object: tony

```
name: Tony Stark  
customerId: 1001  
isActive: true
```

instance

Which type of constructor should be used?



Java Default Constructor

Java provides a default constructor if none is defined

Helps ensure every new object has a predictable initial state

The default constructor executes automatically upon object creation

Its primary purpose is initializing object attributes to default values

```
public class Customer {  
  
    // Explicitly declaring the no-arg constructor  
    public Customer() {  
  
    }  
}  
  
// Calls the default constructor  
Customer c = new Customer();
```

Parameterized Constructor

Parameterized Constructor

- Takes parameters to initialize the object at creation time
- Uses the 'this' keyword to separate the attribute from the parameter
- Replaces the default no-argument constructor once declared

```
public Customer(String name, int customerId) {  
    this.name = name;  
    this.customerId = customerId;  
}
```



```
//parameterized constructor  
public Customer(String name, int customerId,  
                String email, boolean isActive)  
{  
    this.name = name;  
    this.customerId = customerId;  
    this.email = email;  
    this.isActive = isActive;  
}
```

// Usage:

```
Customer tony = new Customer("Tony Stark", 1001,  
                             "tony@stark.com", true);
```

The 'this' Keyword

Understanding 'this':

- ✓ Refers to the current instance of the object
- ✓ Used when parameter names match attribute names
- ✓ Distinguishes between parameter and attribute

Example:

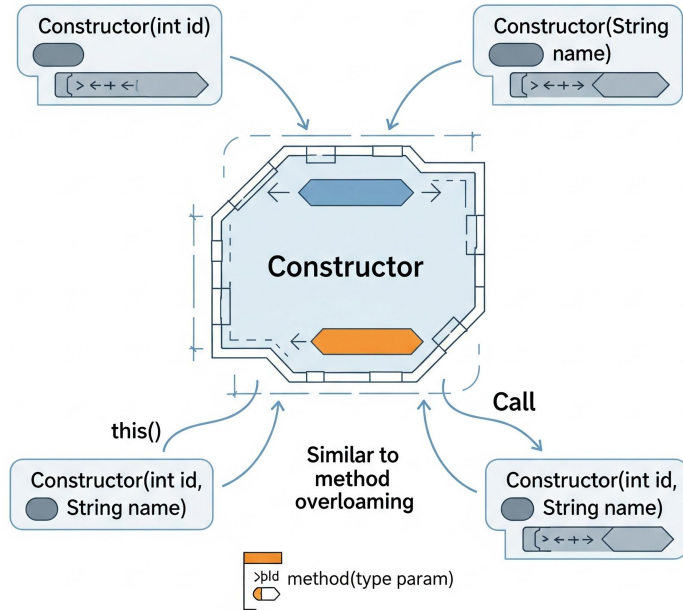
```
this.name = name;
```

Left side: attribute of object

= name

Right side: constructor parameter

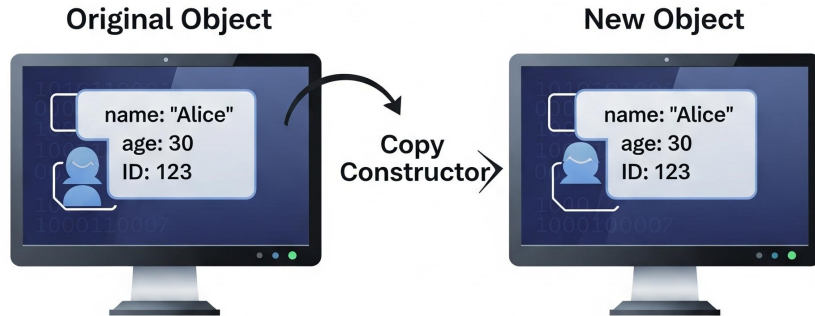
Constructor Overloading



```
// Two-parameter constructor
public Customer(String name, int customerId) {
    this(name, customerId, "not-provided", true);
}

// All-parameter constructor
public Customer(String name, int customerId,
                String email, boolean isActive) {
    this.name = name;
    this.customerId = customerId;
    this.email = email;
    this.isActive = isActive;
}
```

Copy Constructor



```
public Customer(Customer customer) {  
    this(customer.name,  
           customer.customerId,  
           customer.email,  
           customer.isActive);  
}  
  
// Usage:  
Customer bruceClone = new Customer(bruce);
```

Access Modifiers

Access Modifiers for Class Members

- public – accessible from anywhere
- private – accessible only within the class
- protected – accessible within class and subclasses
- _____ – accessible within class and package

Best Practice:

- Keep fields private
- Make methods public



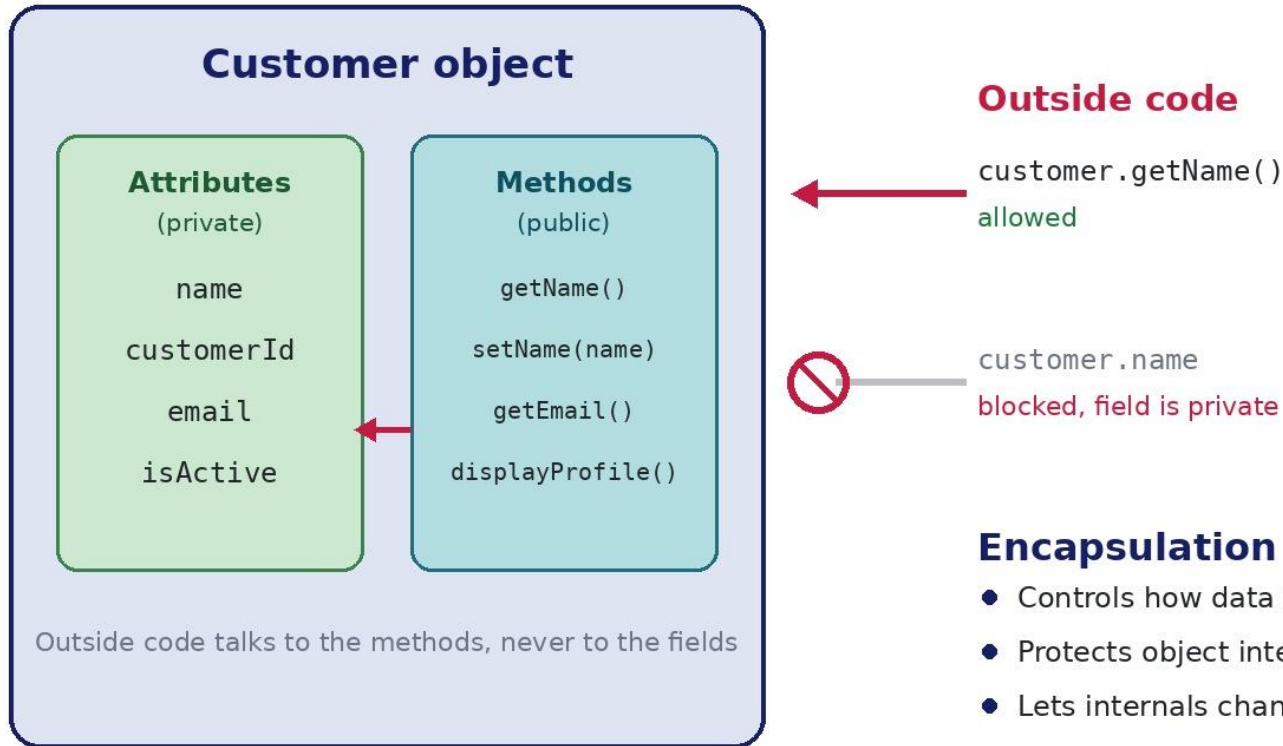
Create a Product class with the following attributes:

- String productId
- String name
- double price
- int stockQuantity

Create the following constructors:

- no-argument constructor
- constructor that takes in productId, name, price and stockQuantity
- constructor that takes in productId, name and price, and sets stockQuantity to 0
- copy constructor

Encapsulation



Encapsulation

- Controls how data is accessed
- Protects object integrity
- Lets internals change safely

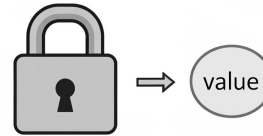
Getters and Setters

```
private String name;

// Getter
public String getName() {
    return name;
}

// Setter
public void setName(String name) {
    this.name = name;
}
```

Getter (Accessor)

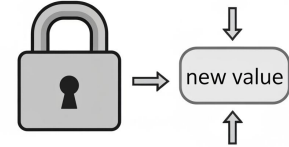


private field

Retrieves the value
of private field

Prefix: "get"
(or "is" for boolean)

Setter (Mutator)

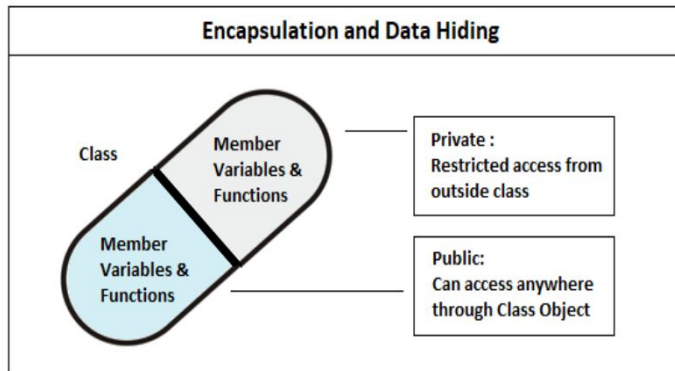


private field

Modifies the value
of private field

Prefix: "set"

Using Encapsulation



Using Encapsulated Fields:

- Cannot access private fields directly
- Must use public getter/setter methods

```
// Make fields private
private String name;
private String email;

// Access using getters
System.out.println(tony.getName());
System.out.println(tony.getEmail());

// Modify using setters
tony.setName("Tony Stark Jr");
tony.setEmail("tony.jr@stark.com");
```

Static Keyword

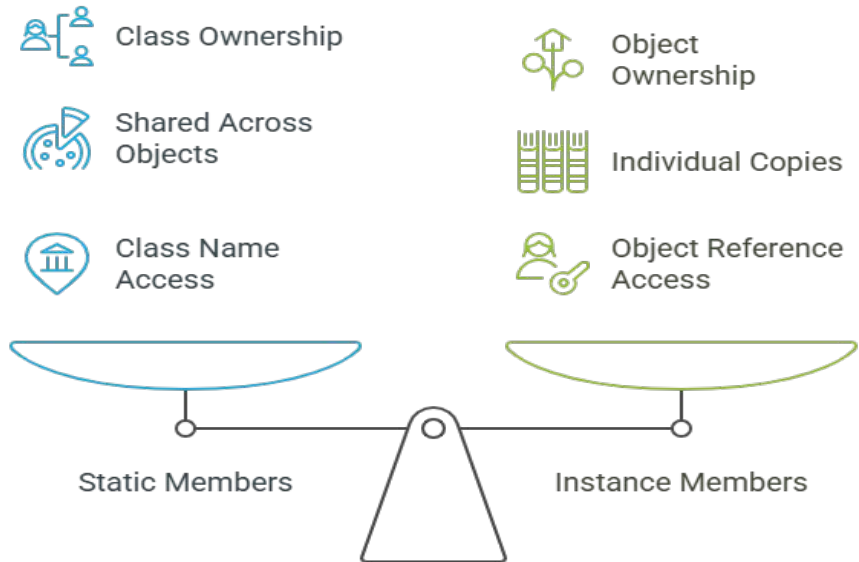
Static Members:

- Belong to the class, not instances
- Shared across all objects
- Accessed using class name

Instance Members:

- Belong to individual objects
- Each object has its own copy
- Accessed using object reference

Compare static and instance members in Java.



Made with  Napkin

Static Attributes

Static Attributes:

- Often used with 'final' keyword
- Can track class-wide information

```
// Static constant
public static final String CUSTOMER_TYPE
    = "RETAIL";

// Static counter
private static int customerCount = 0;

public static int getCustomerCount() {
    return customerCount;
}

// Access using class name
System.out.println(Customer.CUSTOMER_TYPE);
System.out.println(Customer.getCustomerCount());
```

toString() Method

Overriding toString()

- Default toString() prints the class name and hashCode
- Override it to return a meaningful text summary
- Use @Override so the compiler checks the signature for you
- Shows up in logs and stack traces, so keep secrets out

```
@Override  
public String toString() { ... }
```

```
@Override  
public String toString() {  
    return "Customer [name=" + name +  
        ", customerId=" + customerId +  
        ", email=" + email +  
        ", isActive=" + isActive + "];"  
}  
  
// Usage:  
System.out.println(tony);  
// Output: Customer [name=Tony Stark, customerId=1001,...]
```



Continue with the Product class from the previous activity.

1. Encapsulate the attributes and provide accessor and mutator methods.
2. Add a static attribute to keep track of the number of products instantiated and a corresponding static method to get the value.
3. Override the toString method to return a string representation of the object. Then add rules in setPrice and setStockQuantity so negative values are rejected.

POJO (Plain Old Java Object)

What is a POJO?

- **Plain Old Java Object**
 - Simple object to represent data
 - Has private attributes
 - Has constructors
 - Has getters and setters
 - May override toString()
- **Use Cases**
 - Represent data models
 - Pass data between components
 - Store data in collections

Creating a POJO:

- Define private fields
- Add constructors
- Generate getters and setters

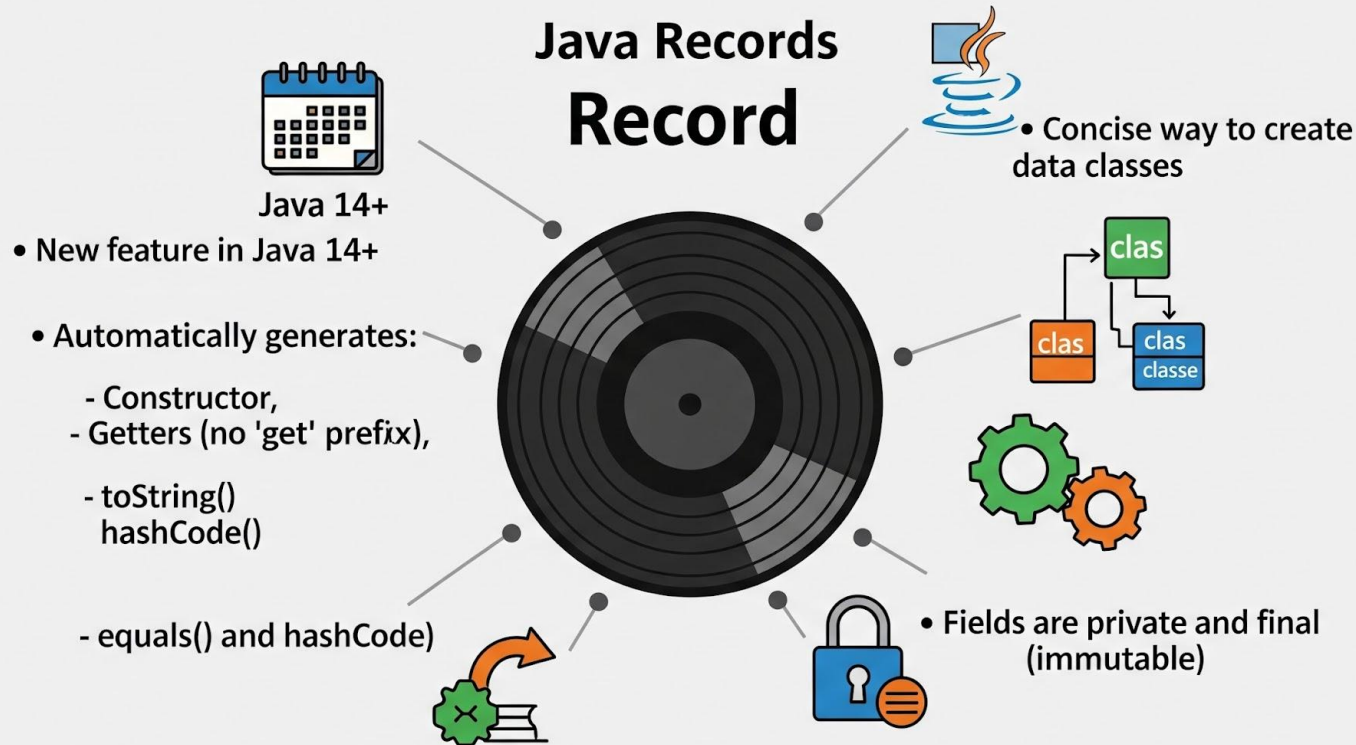
```
public class OrderItem {  
    private String productName;  
    private double amount;  
  
    public OrderItem(String productName, double amount) {  
        this.productName = productName;  
        this.amount = amount;  
    }  
  
    // Getters and setters  
    public String getProductName() { return productName; }  
    public void setProductName(String productName) {...}  
    // ... more getters/setters  
}
```

Using POJOs in Collections:

- Create objects easily
- Store in ArrayList or other collections
- Better than managing separate arrays

```
ArrayList<OrderItem> items = new ArrayList<>();  
  
items.add(new OrderItem("Laptop", 1200.00));  
items.add(new OrderItem("Monitor", 350.00));  
  
// Remember to override toString()  
// to see meaningful output  
System.out.println(items);
```

Record (Java 14+)



Record Example

Creating a Record

- One line of code!



```
<{<-->}>  
<{-->}>
```

-  No getters/setters needed
~~get~~
~~set~~
-  Access fields using `fieldName()` instead of `getFieldName()`
- `getFieldName()` →



```
// Define record - that's it!  
public record OrderItem(String productName, double amount) {}  
  
// Usage:  
OrderItem item = new OrderItem("Laptop", 1200.00);  
  
// Access using field name as method  
System.out.println(item.productName());  
System.out.println(item.amount());  
  
// toString() automatically generated  
System.out.println(item);
```

POJO



Plain Old Java Object

- More verbose (lots of boilerplate code)
- Fields are mutable (can be changed)
- Full control 🔍
- Full control over implementation

Record



- Very concise (minimal code)
- Fields are immutable (cannot be changed)
- Limited customization
- 🔒 Best for simple data carriers

Lesson Summary

What We Covered:

- Classes and Objects (blueprints and instances)
- Constructors (parameterized, overloaded, copy)
- Encapsulation (private fields, getters/setters)
- Static vs Instance members
- toString() method override
- POJOs (data modeling classes)
- Records (concise immutable data classes)